

Design and Implementation of an xApp-based System for Jamming and Interference Mitigation

Aurora Paz-Pérez
Advanced Comm. Dept.
Gradiant
Vigo, Spain
apperez@gradiant.org

Julio Suárez Gómez
Advanced Comm. Dept.
Gradiant
Vigo, Spain
jsuarez@gradiant.org

F. Javier Valera Sánchez
Advanced Comm. Dept.
Gradiant
Vigo, Spain
fvalera@gradiant.org

J. Joaquín Escudero-Garzás
Advanced Comm. Dept.
Gradiant
Vigo, Spain
jescudero@gradiant.org

Abstract—Security is one of main concerns in communications, and radio devices and nodes can be easily targeted through jamming attacks even considering the sophisticated security features of 5G systems. This work presents a comprehensive system based on O-RAN that monitors the spectrum, detects the jamming signal, classifies it, and provides mitigation actions. At the core of our system is an xApp suite that interconnect four xApps, each of them implementing one functionality. We have validated the performance of the xApp suite by implementing it in a 5G O-RAN testbed based on open source libraries and general purpose hardware.

Index Terms—Open RAN, xApp, artificial intelligence, interference detection and mitigation, 5G/6G

I. INTRODUCTION

Cellular networks play a key role in today's society connecting billions of people, services, and critical infrastructure. In this context, a crucial aspect for current and future cellular networks resides in addressing a wide array of security challenges within an increasingly complex threat landscape. Considering the components of a cellular network, user equipment (UE), radio access network (RAN) and core network (CN), RAN is probably the most vulnerable component, due to its inherent over-the-air nature. Among the physical layer threats to the RAN, interference signals are the most relevant, whose effects may range from mild network performance degradation to complete denial of the service. In particular, jamming, or intentional interference, is particularly damaging as the attacker can plan intended network disruptions with cheap, powerful devices. Within jamming, we can distinguish i) smart jamming/non-smart jamming, whether the jammer targets a specific channel or not; ii) time-based jamming, as per the time-domain characteristics of the jamming signal (continuous, periodic, on-off); iii) frequency-based jamming, as per the frequency-domain characteristics of the jamming signal (wideband, narrowband, localized) [1].

Open RAN represents a new paradigm based on open interfaces that offers a disaggregated architecture based on COTS (commercial-off-the-shelf) equipment and introduces radio intelligent controllers (RIC) for a more precise control of the radio components of the network. O-RAN is increasingly gaining traction in 5G and it is expected to keep growing for

the future 6G, due to its flexibility, future-proofing and vendor independence. In addition, the near-Real Time RIC (near-RT RIC) and non-RT RIC entities facilitate the introduction of third-party applications (xApps and rApps, respectively) to optimize the RAN with novel techniques such as Artificial Intelligence (AI), and it is expected that xApps play a key role to detect and mitigate different types of jamming.

When addressing jamming, three aspects are essential: detection, classification, and mitigation. Regarding detection mechanisms, autoencoders are very effective in anomaly detection of jamming attacks by learning and reconstructing normal signals. In [2], the authors propose an autoencoder model that achieves detection accuracy exceeding 90%, outperforming methods like Random Forest and Isolation Forest. In terms of classification, the convolutional neural networks (CNN) approach can achieve effective differentiation among point, periodic, and continuous jamming, with accuracy ranging from 70% to 90% [3], or be effectively combined with K-Nearest Neighbors (KNN) to identify different types of attacks [4].

However, the integration of advanced machine learning (ML) techniques with O-RAN systems via xApps enables a joint approach that may encompass real-time detection, characterization, and mitigation. The authors of [5] present a comprehensive framework for identifying vulnerabilities at the physical layer and propose ML-based methods for early detection of interference/jamming attacks. In [6], the integration and experimental validation of specialized xApps is demonstrated (e.g. the KPIMON xApp [7]), enabling the collection of key performance metrics via the E2 interface for real-time monitoring and anomaly detection. Additionally, the work of [8] presents a statistical method for downlink jamming detection and its implementation in an O-RAN xApp.

In this work, we present the design and implementation of an xApp-based system for jamming and interference mitigation that leverages the benefits of the O-RAN architecture in terms of flexibility and cost. The system consists of a set of AI-based xApps that operate coordinately and efficiently to detect and mitigate different types of jamming attacks. The xApp-based system has been developed into FlexRIC [9], which provides a robust, open source framework for addressing physical layer security in 5G networks. At the same time, it enables dynamic configuration and adaptation,

proactively responding to emerging needs and changes. The integration of ML models not only enhances the detection and precise characterization of jamming attacks but also enables the implementation of adaptive mitigation measures in real time, thereby paving the way for a comprehensive end-to-end solution to emerging threats in wireless communications.

This paper is organized as follows. Section II presents the system architecture and the functionalities of the xApps. Section III shows the testbed implementation and Section IV reports the performance evaluation and validation of the system. Finally, Section V provides our insights.

II. SYSTEM ARCHITECTURE AND FUNCTIONALITIES

This section presents the architecture of our custom O-RAN system and the functionalities (xApps) that enable network monitoring against jamming attacks.

A. Proposed O-RAN Architecture

Our high-level based architecture, shown in Figure 1, focuses on connecting both regular gNBs and those affected by jamming attacks (Jammed Base Station), all linked directly to the CN. At the heart of our infrastructure is the Near-RT RIC, which serves as a critical operational core and hosts the developed xApps. Additionally, the jammer is designed to simulate attacks on the base stations, enabling testing and enhancement of the robustness of our security measures in a controlled environment. In this environment, we have integrated advanced deep learning algorithms, crucial for interference detection and classification to perform the selection of appropriate countermeasures. Through these models, the system can identify and mitigate potential interference attacks.

B. High-level component functionality

This section presents the functionality of the most relevant components at a high level. The proposed system includes two gNBs, both connected to the RIC. This enables remote monitoring and control of the RAN. To simulate the proposed use case, one gNB, which will have a UE connected, will be experiencing interference from a jammer while a more distant one will not be facing this disruptions, as shown in Figure 1. The UE will be the object of interest to optimize its communication link with the RAN. Potential interference is analyzed and appropriate mitigation strategies are implemented to keep the channel quality at the UE. The jammer plays a pivotal role in simulating realistic attacks, which are crucial for validating the detection, classification, and mitigation strategies. The proposed setup allows to assess the resilience of the network against different types of jammers with varying characteristics in the time and frequency domain (see Section II-C).

To enable interference detection and mitigation functionalities, four xApps are implemented in the near-RT RIC, enabling a fast reaction to potential jamming signals. Specifically, we design and implement four xApps with differentiated functionalities that allow to conform the complete flow to analyze the network, detect and classify possible interferences and apply countermeasures if required. The xApps developed are:

- **Monitoring xApp:** Extracts metrics from both jammed and unjammed gNB to analyze the impact of interference.
- **Detection xApp:** Uses ML techniques to detect the presence of jamming as an anomaly in the radio channel.
- **Classification xApp:** Classifies the type of jamming based on the detected characteristics to tailor the mitigation strategy effectively. It considers both frequency and time constraints of the received metrics.
- **Mitigation xApp:** Implements mitigation strategies that are appropriate for the type of jamming identified, sending control actions to the RAN. These actions can change the resources of the cell, controlling UE data rates to strengthen communications or forcing a hard handover.

The xApps need to exchange information in cascade to balance the processing load. To ensure RT communication, ZeroMQ [10] is employed as the messaging tool, providing asynchronous communication with minimal latency, which is a critical aspect in a near-RT environment such as an O-RAN network. Thus, the flow that follows the xApp suite is:

- 1) **Metrics Collection:** The Monitoring xApp continuously updates network metrics at regular intervals of 1 second. When requested, it responds with the most recent data, such as CQI, SNR PUSCH, and SNR PUCCH, all packaged in a JSON format through a ZeroMQ socket.
- 2) **Jamming Detection:** The Detection xApp receives these metrics and processes them using pre-trained ML models to detect anomalies indicative of jamming. Upon detection, the xApp publishes the data along with a jamming detection indicator via the socket.
- 3) **Jamming Classification:** The Classification xApp, subscribing to the Detection xApp's publications, retrieves the anomaly data and applies specialized algorithms to classify the type of jamming, sending through the socket the new information.
- 4) **Mitigation:** The Mitigation xApp request updated jamming classification details. Based on the received information, it coordinates mitigation actions to maintain optimal network performance.

To summarize, our high-level architecture connects regular gNBs and those affected by jamming to the CN, with a jammer simulating attacks to test system resilience and xApps to detect, characterize and mitigate these attacks.

C. xApps Algorithms Design

As explained above, the design of the algorithms implemented in the xApps are developed to ensure effective real-time data collection and prompt jamming detection. The following subsections describe each xApp.

1) *Monitoring xApp:* This xApp operates continuously, extracting real-time metrics from base stations via the E2 interface under both normal conditions and during jamming attacks. The data collected by the KPM Service Model (SM) is always associated with an active UE. The xApp gathers data on signal quality (CQI) and the Signal-to-Noise Ratio (SNR) of both the Physical Uplink Shared Channel (PUSCH) and

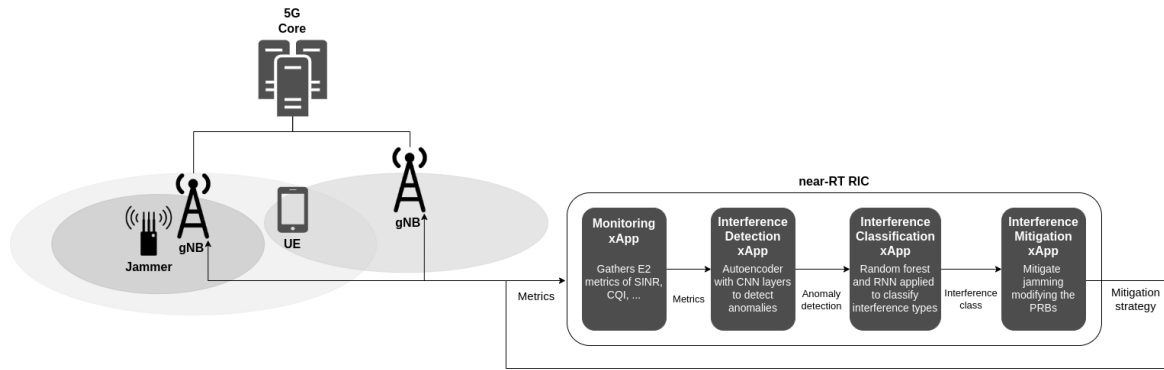


Fig. 1. Proposed O-RAN high-level architecture

the Physical Uplink Control Channel (PUCCH) from the gNB. This data collection is critical for identifying abnormal patterns of behavior in the network that may indicate the presence of jamming or other interferences. It subscribes to periodic cell network measurements from the Centralized Unit (CU) over the E2 interface and forwards these measurements to the Detection xApp for further processing.

2) *Detection xApp*: After the reception of the monitored metrics, the Detection xApp processes the data using ML techniques to detect the presence of jamming. It employs Autoencoders [11] to analyze the data, with the primary goal of differentiating between normal background noise and interference signals for quick and accurate identification of potential jamming attacks. The model consists of two main components:

- **Encoder**: Compresses the input network metrics into a lower-dimensional latent representation that captures the essential features of normal network behavior.
- **Decoder**: Reconstructs the original network metrics from the latent representation.

The difference between the original input and its reconstruction is measured using the Mean Squared Error (MSE), referred to as reconstruction error. Under normal operating conditions, the reconstruction error remains low because the model is trained on baseline network data. However, when jamming interference occurs, the input metrics deviate from the norm, causing a significant increase in the reconstruction error. A predefined threshold established during a calibration phase with baseline data is used to determine if a sample is anomalous. If the reconstruction error exceeds this threshold, the system flags the corresponding data as anomalous, indicating the potential presence of jamming. A continuous learning approach is also incorporated to keep the model updated with evolving network conditions. Periodic retraining ensures that the Detection xApp maintains an up-to-date understanding of normal network behavior.

3) *Classification xApp*: Once interference is detected, the Classification xApp categorizes the jamming type based on the spectrum, intensity, and duration. The classification decides whether the jamming is point/periodic/continuous and localized/narrowband/wideband. This facilitates the jammer

classification into 2 sub-classifiers that independently consider the time and frequency characteristics of the jammer. The Classification xApp will receive all the metrics, whether they are jamming or not. This is because classification is performed in two separate processes:

- **Frequency classification**: Evaluates data on a sample-by-sample basis. The granularity of the sample is determined by the KPM SM. That is, if the RAN is monitored every second, the system provides a new classification that will be mitigated almost every second. Thus, when a sample labeled as jamming by the Detection xApp arrives, it is sent to the frequency classifier, represented as a Random Forest model [12]. In this case we will consider frequency aspects differentiating the type of jammer faced by the RAN between localized/narrowband/wideband jamming.
- **Temporal classification**: This classifier evaluates the data in temporal windows to study the behaviour of the jammer over time. It is considered a temporal window of 10 seconds, value that can be adjusted. This value represents a trade-off as it is large enough to correctly evaluate the type of temporal jammer and not so large to delay mitigation. Within this xApp, a Recurrent Neural Network (RNN) [13] is used to perform the classification. All data is considered by the model whether jammer has been detected or not. In this case, to differentiate the jammer between the different temporal types we have to consider all the metrics within the temporal window. In this way we can classify between the following classes: None/Point/Periodic/Continuous.

By separating the classification into two stages, we achieve higher granularity in the results as each classifier deals with one aspect of the jammer signal. Another advantage is that we can perform faster mitigation since the frequency classifier acts sample by sample while the temporal classifier infers the class considering a larger time window. We can then apply a first mitigation in the frequency domain and apply a second stage for the time domain. Otherwise, we need to wait for the temporal classifier to provide its inference to apply joint mitigation. Therefore, we lower the system performance latency, becoming more reactive to external interference.

4) *Mitigation xApp*: The last xApp in the workflow is the Mitigation xApp. It implements specific mitigation strategies designed to counter the identified type of jamming using the information provided by the Classification xApp. These actions are included in the RAN Control (RC) SM and are related with control changes in the configuration of the gNB and UE. Therefore, we can apply algorithms that allow, based on obtained data, an optimization of the RAN. These actions are limited by the RC SM, so it is not possible to change all the desired parameters of the network, but only those that are indicated in the standard and that are also implemented by both gNB and near-RT RIC libraries. In this case, the variable to be modified is the number of Physical Resource Blocks (PRB) assigned to the UE. By changing the number of PRBs, we force the UE to perform a hard handover to another gNB if possible, as it will have better performance since more resources could be allocated to it. If this change is not possible because there is no gNB nearby the UE would regulate its traffic, lowering the rate. The PRBs are then allocated according to the class received so that a first adjustment is applied when the frequency class is received. Subsequently, a second adjustment is performed according to the temporal class.

These are all system elements that enable monitoring and mitigation of jamming attacks within an O-RAN architecture.

III. SYSTEM IMPLEMENTATION

This section will focus on the implementation of the different blocks described above. More specifically, we focus on the xApp suite deployed on the O-RAN architecture (see Fig.2) that includes servers, SDRs, and RF connectivity.

To implement the CN, we use the open source library Open5GS version 2.4.9 [14]. The near-RT RIC is deployed through the FlexRIC library, commit a79610c3 [9] and the two gNBs (RAN E2 nodes in the figure) are deployed through srsRAN release 24.04 [15]. Both gNBs implement the O-RAN split 7.2 and connect to the RIC through the E2 interface. The UE is implemented using srsRAN through srsUE release 23.11 [16]. This emulated UE facilitates the deployment automation of the complete testbed and experimenting with the xApp suite in a simple way. Deployments are done using Docker Compose as Docker [17] containers were created for all elements.

For the RF part, the utilization of USRP B210 [18] provides an easily reprogrammable system, being able to adjust through the configuration files parameters such as the operating band, gains, and rate. Moreover, the use of the interconnection matrix is relevant in this setup since, due to the physical space of the laboratory, we cannot set up a network that allows a realistic handover. Therefore, we can define the desired attenuations between the different equipment, simulating that one gNB is at a greater distance than another. In this way the UE will connect to one gNB but, subsequently, due to the mitigation applied caused by jammer detection, a handover to the other gNB will be performed.

The jammer is implemented using GNU Radio [19] and is deployed on a USRP B210. The designed jammer supports

multiple interference scenarios by transmitting different types of jamming signals. In particular, the jammer can simulate:

- **Frequency Characteristics:**

- *Wideband*: Affects a broad range of frequencies.
- *Narrowband*: Affects a very narrow frequency range.
- *Localized*: Targets specific frequencies.

- **Temporal Characteristics:**

- *Continuous*: Jamming occurs constantly.
- *Periodic*: Jamming occurs periodically for fixed time.
- *Intermittent*: Jamming occurs in bursts, testing the system's recovery and response capabilities.

The combination of the above results in nine distinct jamming types. This flexible design makes the jammer a powerful tool for testing the resilience and our mitigation strategies.

Network connectivity is also required among servers and the Internet. For that, a firewall and a switch are used to enable communication among the different software elements, as shown in Fig.2. The distribution of the elements allows load balancing between the processes that require higher CPU consumption, namely gNB, UE and the xApps.

Once the entire system is deployed, the xApps are executed to perform the main logic of the system as indicated above, being able to run the validation of the complete system.

IV. RESULTS

Our primary objective is to validate the performance and accuracy of the xApps deployed within the near-RT RIC under the different interference conditions described in upper sections. The laboratory environment is designed to emulate realistic O-RAN network conditions. Overall orchestration ensures consistent deployment across all components, allowing real-time validation of data collection, anomaly detection, dual-mode classification, and dynamic mitigation.

A. Detection xApp evaluation

The detection tests were conducted to assess the performance of the autoencoder-based Detection xApp under various jamming scenarios. In these tests, the xApp processes SNR values from the PUSCH and PUCCH channels sent by the Monitoring xApp. The autoencoder, pre-trained on data obtained during normal operation, reconstructs the input and computes the reconstruction error via the MSE.

To comprehensively evaluate the detection mechanism, the experiments cover the nine jamming scenarios reported in Section III that combine both the frequency and temporal domains. Table I summarizes the experimental results of these scenarios, where MSE indicates the error in normal conditions and MSE(J) is the error under jamming attack. For localized jamming, the measured MSE values for normal operation were consistently below approximately 0.015, whereas under jamming conditions MSE increases substantially (up to 1.27 in the continuous case). We observe similar trends for the narrowband and wideband scenarios, with detection rates close to 100% and only minor discrepancies in false negatives, demonstrating that the detection mechanism reliably distinguishes abnormality under different interference conditions.

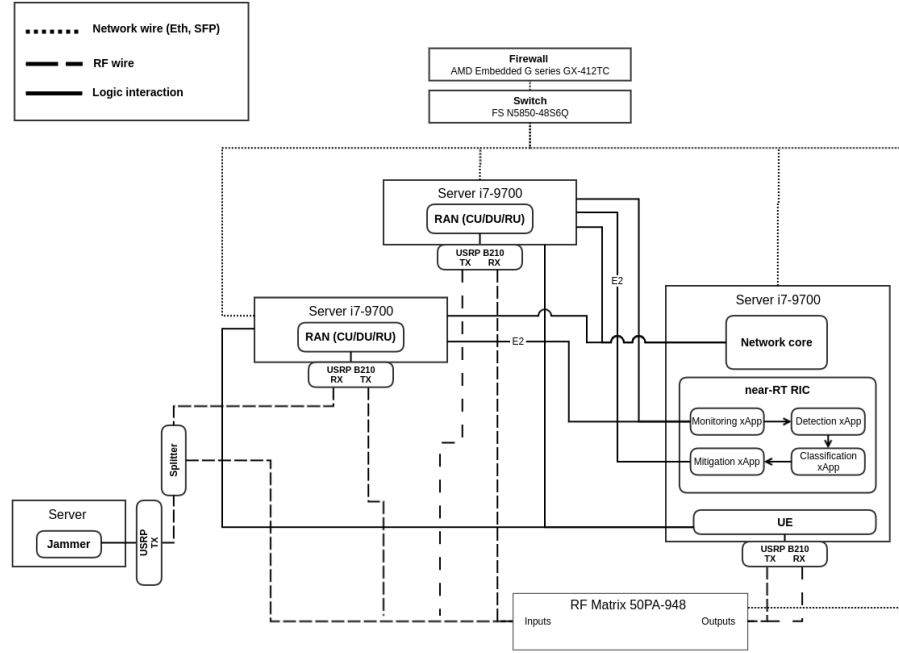


Fig. 2. System implementation architecture

The consistency of the values across all nine scenarios underscores the reliability of the detection approach in a laboratory environment that emulates a realistic O-RAN network.

TABLE I
DETECTION TEST RESULTS FOR JAMMING SCENARIOS

Test	Rate	MSE	MSE (J)
Localized Point	98%	1.3860×10^{-2}	3.6261×10^{-2}
Localized Periodic	100%	1.1786×10^{-2}	5.3023×10^{-1}
Localized Continuous	100%	1.4084×10^{-2}	1.2722×10^0
Narrowband Point	100%	6.8684×10^{-5}	2.1656×10^{-1}
Narrowband Periodic	100%	1.9293×10^{-4}	1.0176×10^0
Narrowband Continuous	100%	1.7982×10^{-4}	4.0842×10^{-2}
Wideband Point	100%	1.2897×10^{-4}	1.9310×10^{-1}
Wideband Periodic	100%	1.2897×10^{-4}	1.9310×10^{-1}
Wideband Continuous	100%	1.7249×10^{-4}	2.4371×10^0

B. Classification xApp evaluation

To ensure proper functioning of the Classification xApp, a minimum accuracy threshold of 0.85 is set to ensure that the system can reliably distinguish between different types of jamming. As indicated above, the classification is divided into two dimensions, frequency and time, with independent classifiers. Consequently, the training and evaluation are differentiated.

The performance of the Classification xApp is shown in Fig.3 for the Random Forest algorithm in the frequency domain. The algorithm classifies the metrics well 100% of the time because the classification in the frequency domain is very straightforward once the Detection xApp has reported an anomaly, largely exceeding the minimum threshold of 85%.

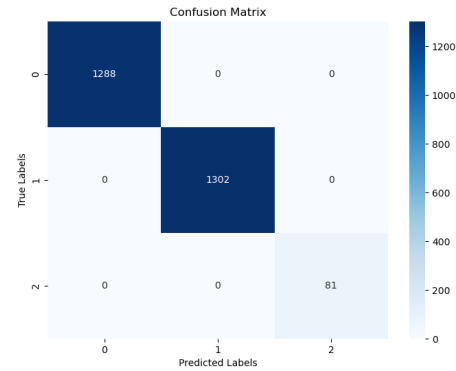


Fig. 3. Frequency domain. Confusion matrix of the Random Forest algorithm

On the other hand, Fig.4 shows the results for the time domain classification, which is performed using a RNN trained on 25 epochs with a batch size of 32. The data used in training corresponds to approximately 3.5 hours of metrics.

Table II reports the results for each time-domain jamming class. We observe that the Continuous class is the one with the best metrics, followed by No Jamming. This is due to the fact that they are easy to differentiate, as they behave as an on-off interference all the time. On the other hand, some crossover is generated between the Punctual and Periodic classes. This is due to the fact that, realistically, these types of jammers do not always have the same period, so that a periodic jamming can become similar to punctual one and viceversa due to the

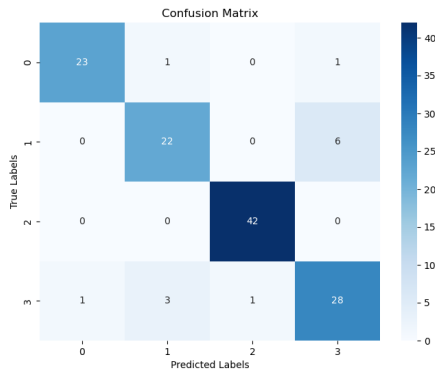


Fig. 4. Time domain. Confusion matrix of the RNN algorithm

TABLE II
METRICS OBTAINED WITH THE RNN ALGORITHM

	No Jamming	Punctual	Continuous	Periodic	Mean
Accuracy	0.98	0.92	0.99	0.91	0.95
Recall	0.96	0.85	0.98	0.80	0.90
Precision	0.92	0.79	1.00	0.85	0.89
F1-score	0.94	0.81	0.99	0.82	0.89

delays of the physical devices. In spite of this, the performance is very satisfactory with an average of 0.95 accuracy.

These tests verify the correct operation of the classification algorithms introduced in the O-RAN architecture.

C. Mitigation xApp evaluation

To perform the mitigation tests, 2 gNBs are required for handover purposes, and the maximum latency threshold for the complete system is set to 5 seconds. As explained in Section III, there are two phases in the mitigation process to ensure the defined threshold. The measurement of the total latency through the xApps starts when the Monitoring xApp receives metrics and concludes when the Mitigation xApp receives the ACK from the RAN indicating the correct application of changes. The maximum delay that the sample reception can suffer since the jammer is active is 1s given that the values are taken every second (and then processed by the Detection xApp and Classification xApp). To obtain the maximum and minimum delay, the jammer is launched either just after obtaining a sample or just before a sample is taken. A set of 200 delay measurements is used for the evaluation and we obtain a maximum delay of 3.2s, a minimum delay of 1.14s, and a mean delay of 2.17s.

Therefore, the delay KPI is met as well as the rest of the KPIs, demonstrating the correct functioning of all xApps and the benefits of these for anomaly detection, jamming classification and jamming mitigation.

V. CONCLUSIONS

This document describes the design of an xApp-based system for jamming and interference mitigation and its implementation on a testbed architecture that enables experiments validating the performance. The system leverages the

advantages of O-RAN through the metrics and actions enabled by the KPM and RC Service Models of the E2 interface to design IA-based and custom algorithms implemented in the collaborative xApps. The metrics outline the xApp system efficiency in detecting, classifying and mitigating different types of jamming interference. Future research work will be focused on extensions to the O-RAN E2 interface.

ACKNOWLEDGMENT

This work was supported by the MINECO (Spain) and the EU NextGenerationEU/PRTR (Call UNICO I+D 5G 2021, ref. number TSI-063000- 2021-13 – 6GENABLERS-SEC).

REFERENCES

- [1] P. Lohan, B. Kantarci, M. Amine Ferrag, N. Tihanyi, and Y. Shi, "From 5g to 6g networks: A survey on ai-based jamming and interference detection and mitigation," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 3920–3974, 2024.
- [2] O. T. Başaran, M. Başaran, D. Turan, H. G. Bayrak, and Y. S. Sandal, "Deep autoencoder design for rf anomaly detection in 5g o-ran near-rt ric via xapps," in *2023 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2023, pp. 549–555.
- [3] Z. Wu, Y. Zhao, Z. Yin, and H. Luo, "Jamming signals classification using convolutional neural network," in *2017 IEEE International Symposium on Signal Processing and Information Technology (ISSPIT)*. IEEE, 2017, pp. 062–067.
- [4] S. Lefcourt, N. Gordon, H. Wong, and G. Falco, "Space cognitive communications: Characterizing radiofrequency interference to improve digital space domain awareness," in *2022 International Conference on Localization and GNSS (ICL-GNSS)*. IEEE, 2022, pp. 1–7.
- [5] M. S. J. Solajja, H. Salman, and H. Arslan, "Towards a unified framework for physical layer security in 5g and beyond networks," *IEEE Open Journal of Vehicular Technology*, vol. 3, pp. 321–343, 2022.
- [6] J. Groen, S. D'Oro, U. Demir, L. Bonati, M. Polese, T. Melodia, and K. Chowdhury, "Implementing and evaluating security in o-ran: Interfaces, intelligence, and platforms," *IEEE Network*, 2024.
- [7] OAIC, *OAIC Software Community API Documentation*, 2022. [Online]. Available: <https://openaicc.github.io/oaic/index.html>
- [8] P. Kryszkiewicz and M. Hoffmann, "Open ran for detection of a jamming attack in a 5g network," in *2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring)*, 2023, pp. 1–2.
- [9] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: An SDK for next-generation SD-RANs," in *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies*, 2021, pp. 411–425.
- [10] "ZeroMQ," accessed: 2025-04-10. [Online]. Available: <https://zeromq.org/get-started/>
- [11] D. Bank, N. Koenigstein, and R. Giryas, "Autoencoders," 2021. [Online]. Available: <https://arxiv.org/abs/2003.05991>
- [12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [13] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, Mar. 2020. [Online]. Available: <http://dx.doi.org/10.1016/j.physd.2019.132306>
- [14] "Open5GS: Open Source 5G Core and EPC Implementation," accessed: 2025-04-10. [Online]. Available: <https://open5gs.org/>
- [15] "srsRAN: Open Source LTE and 5G RAN Software Suite," accessed: 2025-04-10. [Online]. Available: <https://docs.srsran.com/projects/project>
- [16] "srsUE: software UE modem," accessed: 2025-04-10. [Online]. Available: <https://docs.srsran.com/projects/4g>
- [17] "Docker: A Platform to Build, Run, and Share Applications," accessed: 2025-04-10. [Online]. Available: <https://www.docker.com/>
- [18] a. N. I. B. Ettus Research, "USRP B210: Software Defined Radio," accessed: 2025-04-10. [Online]. Available: <https://www.ettus.com/all-products/ub210-kit/>
- [19] "GNU Radio: A Free & Open-Source Toolkit that Provides Signal Processing Blocks to Implement Software Radios," accessed: 2025-04-10. [Online]. Available: <https://www.gnuradio.org/>